

ARTEFATO 02 — Spec Completa: Elisp × Emacs × Human-in-the-Loop

"O Editor como Sistema Nervoso Central do Processo Vivo"

Série: IA×Energia×Web3 | Artefato Intermediário #02

 **Status:** Spec Arquitetural | Versão 1.0  **Depende de:** Artefato 01 (Outline Filosófico-Técnico)  **Gera:** Artefatos 03–09 (implementações incrementais)

Filosofema de Abertura

"O Emacs não é um editor de texto. É um ambiente de pensamento exteriorizado — um córtex artificial onde o humano e a máquina não se alternam, mas co-habitam." — parafraseando Richard Stallman & Alan Kay

"Elisp não é uma linguagem de configuração. É um Lisp vivo que habita um processo que nunca morre — o daemon mais antigo da computação pessoal."

"Human-in-the-loop não é uma limitação de velocidade. É a sabedoria de saber que alguns gates só devem abrir com presença humana."

0. Por que Emacs/Elisp para este Projeto

0.1 Rationale Filosófico-Técnico




PROBLEMA COM ABORDAGENS COMUNS:

- Python script → stateless, morre, esquece
- Jupyter → interativo mas sem daemon
- Web app → overhead, latência, cloud-first
- CLI daemon → eficiente mas anti-humano

SOLUÇÃO EMACS:

- ✓ Processo que nunca morre (daemon desde 1976)
- ✓ REPL vivo = inspect/modify sem restart
- ✓ Org-mode = documentação É o sistema
- ✓ Human gates naturais (aceitar/rejeitar com C-c C-c)
- ✓ RLHF inline: feedback direto no fluxo de trabalho
- ✓ Buffer = contexto persistente e editável
- ✓ Elisp = código e dados são a mesma coisa (homoiconicidade)

0.2 Rationale Técnico: Emacs como Daemon




emacs --daemon ← processo eterno, igual ao pseudoassembly do A01
emacsclient ← conexão efêmera ao daemon (como MCP client)
emacs server ← persiste estado, buffers, variáveis entre sessões

ISOMORFISMO COM A01:
daemon ↔ emacs --daemon
poll_event() ↔ run-with-timer / file-notify-add-watch
llm_infer() ↔ gptel-request (async)
execute_via_mcp ↔ shell-command / url-retrieve (async)
update_memory() ↔ org-capture / append-to-file
sleep() ↔ (run-with-idle-timer N t #fn)

0.3 Metacrítica: Limitações Conscientes

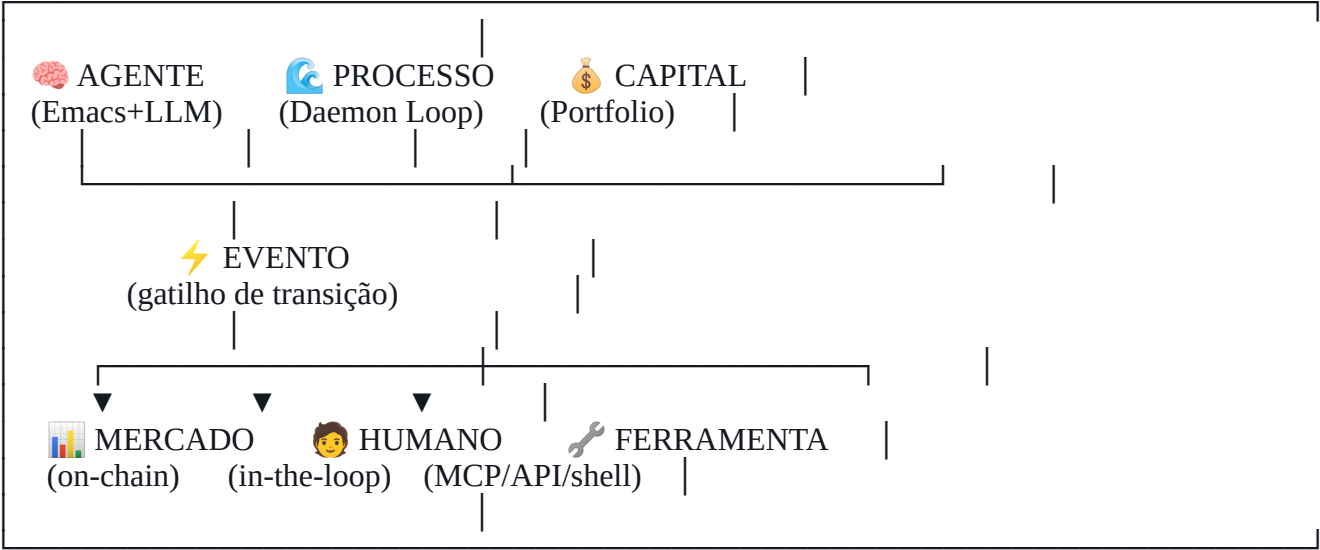
-  **MC-01:** Elisp é single-threaded (GIL análogo). **Mitigação:** usar make-process + sentinelas para I/O paralelo; inferência LLM via processo externo (llama.cpp) com callback async.
-  **MC-02:** Curva de aprendizado do Emacs é alta. **Mitigação:** o projeto já pressupõe usuário técnico; Org-mode oferece UX familiar (markdown-like) para o human-in-the-loop.
-  **MC-03:** Performance de Elisp vs. C para loops intensivos. **Mitigação:** Elisp orquestra; binários externos executam. Divisão clara.

1. Ontologia do Sistema

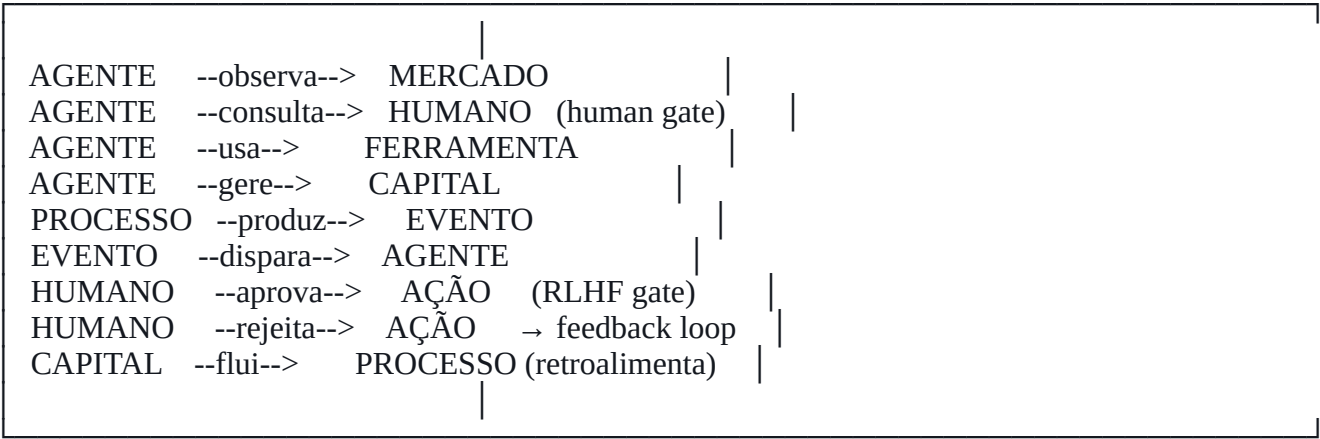
1.1 Diagrama Ontológico (ASCII + Emojis)



ENTIDADES FUNDAMENTAIS



RELAÇÕES



ESTADOS DO CAPITAL (máquina de estados)



1.2 Vocabulário Ontológico Formal



- CONCEITOS PRIMITIVOS:
- Processo := entidade com $F=\emptyset$ (sem estado terminal)
 - Evento := tupla (timestamp, tipo, payload, fonte)
 - Gate := ponto de decisão que requer presença humana
 - Feedback := sinal (positivo|negativo|neutro) sobre ação passada
 - Energia := capital em movimento (não estoque)
 - Contexto := estado comprimido do processo em t

- CONCEITOS DERIVADOS:
- RLHF-inline := feedback humano integrado ao ciclo do daemon
 - Human-gate := Gate com timeout (se humano ausente → escalar)
 - Prompt-vivo := template que se auto-modifica via RAG + feedback
 - Buffer-org := estrutura de dados Emacs = contexto legível+editável

2. AST — Árvore de Sintaxe Abstrata do Sistema

2.1 AST do Sistema (não de código — do domínio)



AST: SISTEMA-IA-WEB3

```
(sistema-ia-web3
  (agente
    (núcleo
      (daemon emacs-server)
      (llm phi3-local)
      (memória org-roam))
    (módulos
      (coletor
        (fontes api-precos api-sentimento on-chain))
      (analizador
        (métricas nvt mvrv mindshare hype-score))
      (decisor
        (estratégias conservadora moderada arrojada)
        (gates human-gate auto-gate))
      (executor
        (canais mcp-server shell-command url-retrieve))
      (avaliador
        (métricas acurácia pnl drawdown)
        (feedback rlhf-inline retroalimentação))))
  (dados
    (entrada eventos-mercado)
    (estado org-buffer mmap-state)
    (saída ordens alertas relatórios))
  (humano
    (interface emacs-ui org-agenda)
    (gates aprovar rejeitar adiar)
    (feedback positivo negativo correção))
  (infraestrutura
    (runtime emacs-daemon)
    (processos llama-cpp make-process)
    (persistência org-files sqlite sqlite-wal)))
```

2.2 AST do Loop Principal em Elisp (pré-código)




```
(daemon-loop
  (init
    (load-state "~/emacs.d/web3-agent/state.org")
    (start-watchers arquivo timer websocket))
  (ciclo
    (await-event
      (on-timer → coletar-dados)
      (on-file → processar-alerta)
      (on-keypress → human-input))
    (inferir
      (construir-prompt contexto evento)
      (chamar-llm-async prompt callback))
    (callback
      (parsear-resposta raw-text)
      (extrair-acao json-schema)
      (verificar-gate ação perfil-risco))
    (gate
      (if auto-aprovado
        (executar-ação ação)
        (apresentar-ao-humano ação buffer-decisão)))
    (human-branch
      (exibir-contexto buffer)
      (aguardar-input "Aprovar [y/n/e(editar)]?")
      (registrar-feedback resposta-humana))
    (pós-ciclo
      (atualizar-memória estado novo-estado)
      (calcular-métricas acurácia ciclo)
      (auto-otimizar se necessário)
      (sleep-until próximo-evento))))
```



3. ADR — Architecture Decision Records

ADR-001: Emacs como Runtime Principal




ADR-001: Usar Emacs Daemon como Runtime do Sistema	
STATUS: Aceito DATA: Sessão #02	
CONTEXTO: Precisamos de um processo persistente que suporte human-in-the-loop nativo, RLHF inline e manipulação de texto estruturado como dado primário.	
DECISÃO: Emacs --daemon como processo principal. Toda lógica em Elisp, processos externos para computação pesada.	
CONSEQUÊNCIAS POSITIVAS: + Processo eterno com estado persistente + Org-mode como formato unificado de dados + Human gates naturais via keybindings + REPL para inspeção/modificação em runtime + Histórico como texto legível (auditabilidade)	
CONSEQUÊNCIAS NEGATIVAS: - Single-threaded (mitigado por make-process) - Overhead de 50-150MB RAM para Emacs - Curva de aprendizado Elisp	
ALTERNATIVAS REJEITADAS: - Python asyncio: stateless entre restarts - Node.js daemon: sem human-in-the-loop nativo - Jupyter: interativo mas não daemon	

ADR-002: Org-mode como Formato Universal de Estado



ADR-002: Org-mode como Estado, Memória e Interface	
STATUS: Aceito	
DECISÃO: Todo estado do sistema é um arquivo .org. Memória, logs, decisões pendentes, histórico — tudo em Org.	
RATIONALE: Org-mode é simultaneamente: → Banco de dados (properties, tags, timestamps) → Interface (agenda, kanban, tabelas) → Log imutável (append-only via org-capture) → Contexto para LLM (texto estruturado legível) → Gate de decisão (TODO/DONE/REJECTED) Filosofema: "O formato dos dados define o pensamento possível. Org-mode permite pensar em processo."	

ADR-003: LLM Local via llama.cpp + gptel



ADR-003: Inferência Local, Orquestração via gptel	
STATUS: Aceito (com revisão em 3 meses)	
DECISÃO: llama.cpp serve Phi-3 localmente via HTTP (OpenAI compat). gptel.el faz chamadas async do Emacs. Fallback: API Claude/OpenAI se hardware insuficiente.	
CRITÉRIO DE REVISÃO: Se acurácia < 65% em 30 dias → escalar para cloud LLM	

ADR-004: Human Gates com Timeout e Escalonamento



ADR-004: Human Gates Temporizados	
STATUS: Aceito	
DECISÃO: Todo gate humano tem timeout configurável. Expirado o timeout: → Perfil conservador: REJEITAR (não agir) → Perfil moderado: MANTER (sem mudança) → Perfil arrojado: EXECUTAR (ação sugerida) Emacs notifica via: minibuffer + org-agenda + (opcional) notificação desktop via D-Bus/notify-send	

ADR-005: RLHF Inline via Org-mode Feedback Entries

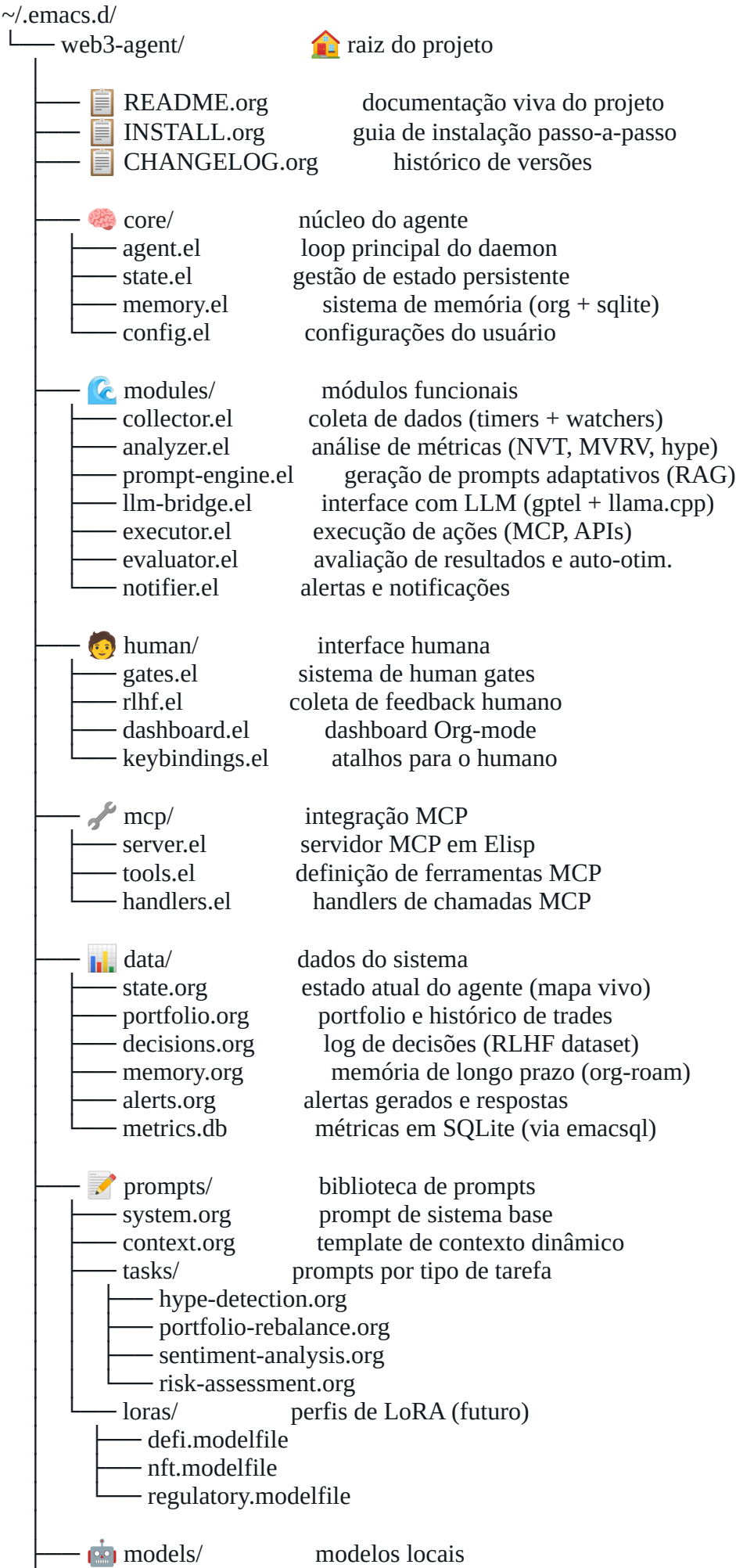


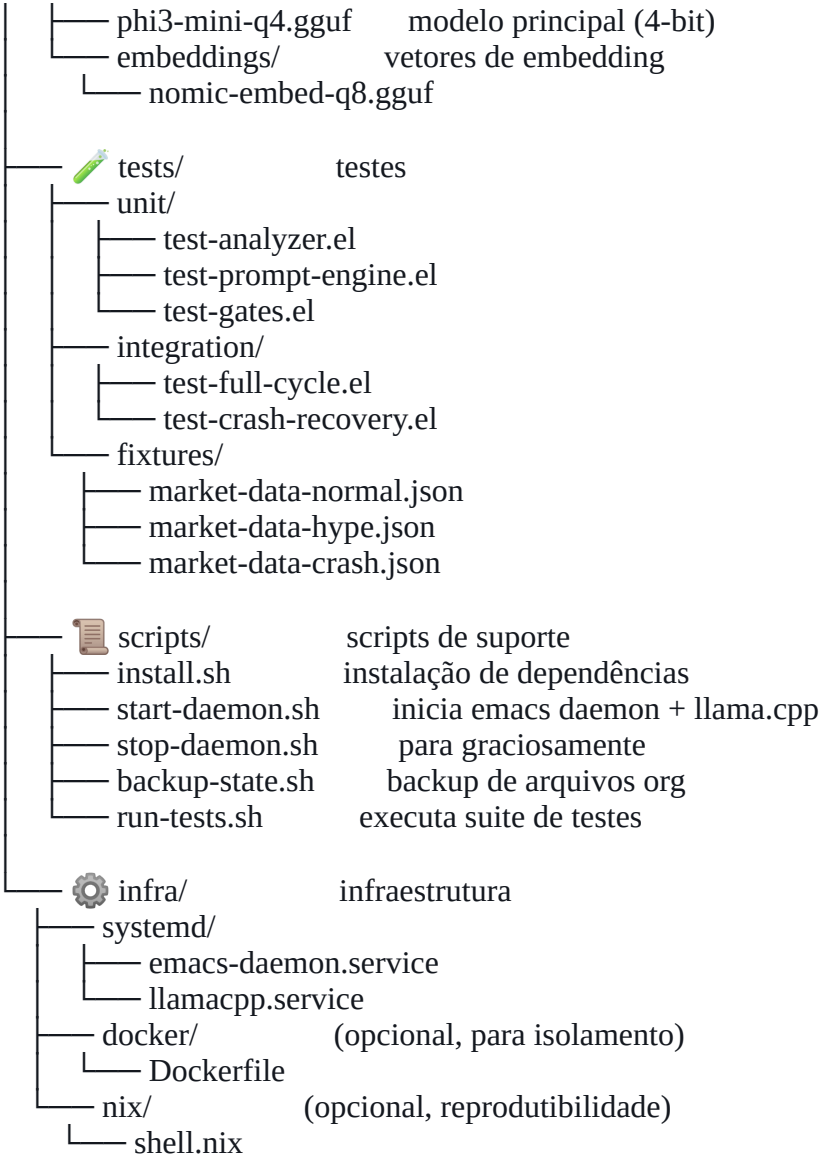
ADR-005: RLHF como Captura Org Estruturada	
STATUS: Aceito	
DECISÃO: Cada decisão humana (aprovar/rejeitar/editar) gera automaticamente uma entrada org-capture com: - Contexto da decisão - Ação sugerida pelo LLM - Ação executada pelo humano - Resultado 24h depois (preenchido pelo daemon) - Rating do humano (1-5 estrelas, C-c C-c) Este arquivo é o dataset de fine-tuning do sistema. Filosofema: "Cada decisão é uma semente de sabedoria."	



4. Estrutura de Pastas & Arquivos



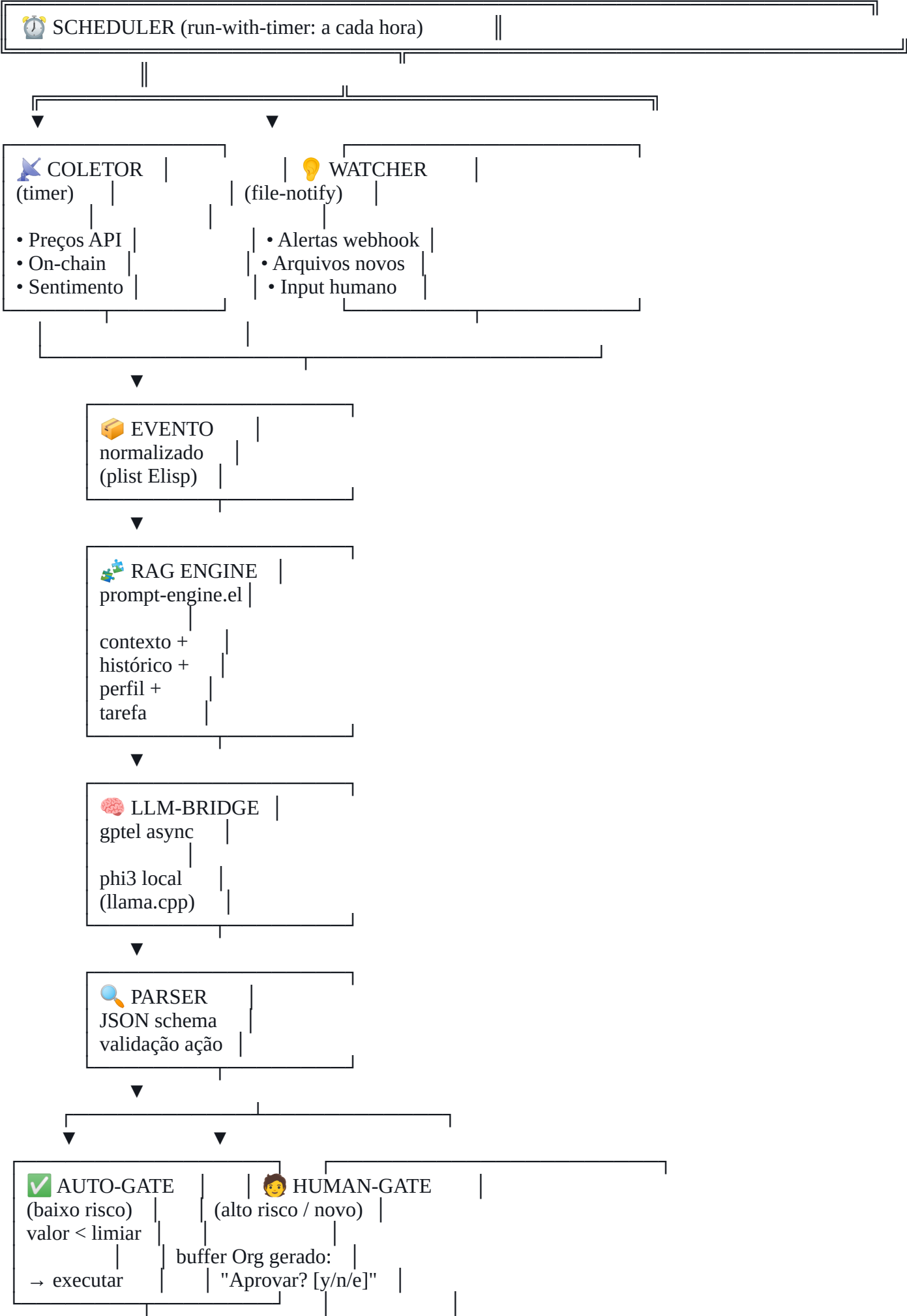


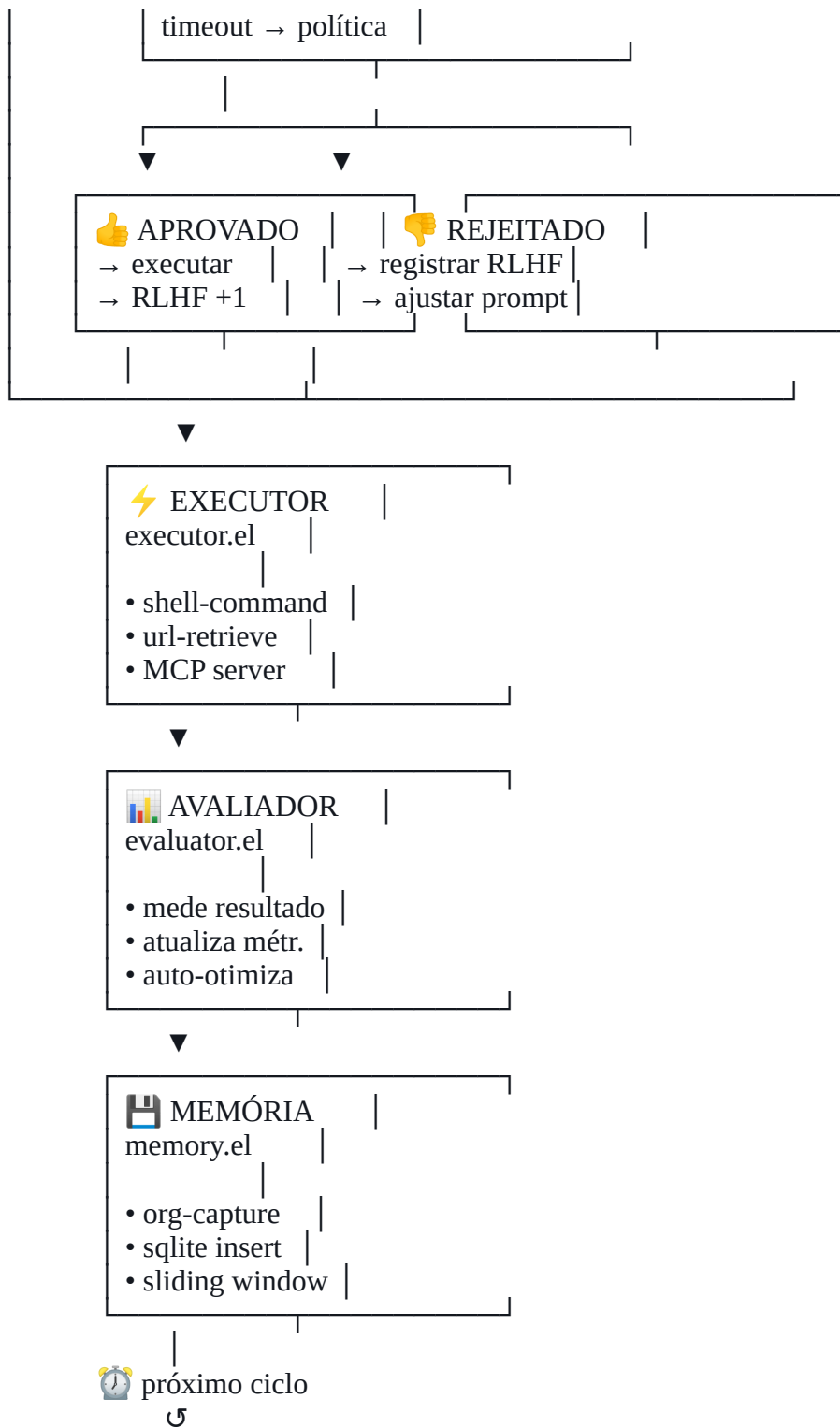


5. Pipeline de Automação

5.1 Pipeline Principal (Mapa Visual)



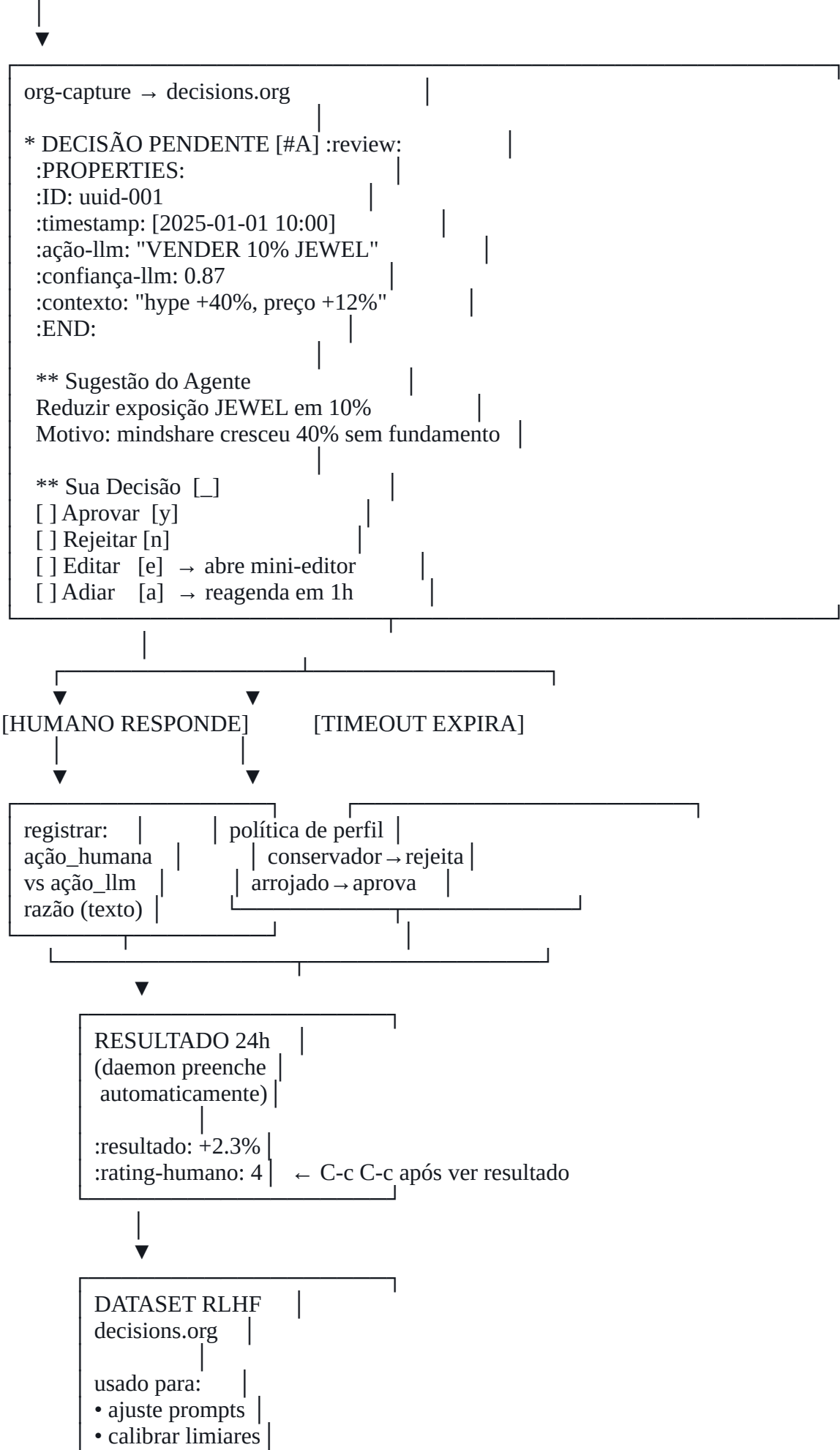




5.2 Pipeline de RLHF (detalhado)



EVENTO DE DECISÃO



6. Workflow Humano

6.1 Rotina Diária do Investidor (Emacs-first)



WORKFLOW HUMANO — DIA TÍPICO

MANHÃ (5 min)

M-x web3-dashboard ← abre dashboard Org

ENERGIA FINANCEIRA — 2025-01-01

\$ Total: R\$ 1.247 (↑ 3.2% 24h)

↻ Em fluxo: R\$ 890 (staking + LP)

🏠 Reserva: R\$ 357 (USDC)

⚠️ ALERTAS PENDENTES (2)

→ [?] JEWEL hype detectado (conf. 87%) [y/n/e]

→ [?] Rebalancear para 15% GameFi? [y/n/e]

✅ EXECUTADOS ONTEM (3)

→ Staking reward coletado (+R\$2.40)

→ LP fee harvest (+R\$0.87)

→ Análise semanal gerada

DECISÕES (2 min)

Cursor no alerta → y para aprovar / n para rejeitar

C-c C-e para editar o valor antes de executar

C-c C-d para ver detalhes completos (contexto LLM)

NOITE (2 min — opcional)

M-x web3-review-decisions

→ ver resultados das decisões de ontem

→ dar rating (1-5) com C-c C-c

→ isso alimenta o RLHF dataset

SEMANAL (15 min)

M-x web3-weekly-report

→ relatório gerado por LLM em Org-mode

→ edite, aprove, exporte como PDF se quiser

M-x web3-tune-parameters

→ ajuste limiares de gate manualmente

→ visualize drift de acurácia do sistema

6.2 Keybindings Principais (Human Interface)



KEYMAP: web3-agent-mode

GLOBAL (disponível sempre que Emacs estiver aberto)

C-c w d	→ web3-dashboard	(abre painel)
C-c w a	→ web3-alerts	(ver alertas pendentes)
C-c w p	→ web3-portfolio	(snapshot do portfolio)
C-c w r	→ web3-report	(relatório LLM)
C-c w s	→ web3-status	(status do daemon)

NO BUFFER DE DECISÃO (gate aberto)

y	→ aprovar ação sugerida
n	→ rejeitar ação
e	→ editar ação antes de executar
a	→ adiar (pede novo horário)
d	→ detalhes (expande contexto LLM)
q	→ fechar sem decidir (= adiar 1h)

NO BUFFER DE REVISÃO (RLHF rating)

1-5	→ rating da decisão passada
C-c C-n	→ adicionar nota de feedback
C-c C-c	→ confirmar rating e fechar

7. Stack de Instalações

7.1 Stack Completo (camadas)



CAMADA 7: APLICAÇÃO (nosso projeto)

web3-agent/ (este projeto)		
org-roam (memória de longo prazo)		
gptel (bridge LLM)		
emacsql + sqlite (métricas)		

CAMADA 6: EMACS ECOSYSTEM

Emacs 29.x+ (com tree-sitter, native-comp)		
use-package (gestão de pacotes)		
straight.el (pacotes de git)		
org 9.7+ (org-mode moderno)		

CAMADA 5: LLM LOCAL

llama.cpp (servidor HTTP OpenAI-compatible)		
phi3-mini-4k-instruct-Q4_K_M.gguf (modelo principal)		
nomic-embed-text-v1.5.Q8_0.gguf (embeddings)		
ollama (opcional, alternativa a llama.cpp)		

CAMADA 4: DADOS & APIs

SQLite 3.x (métricas locais)		
curl / url.el (HTTP requests)		
CoinGecko API (preços — free tier)		
LunarCrush API (sentimento/mindshare)		
Harmony RPC (dados on-chain ONE/JEWEL)		

CAMADA 3: SISTEMA OPERACIONAL

Linux (Ubuntu 22.04+ / Debian 12+) OU macOS 13+		
systemd (gestão do daemon — Linux)		
launchd (gestão do daemon — macOS)		
file-notify / inotify (watchers)		

CAMADA 2: LINGUAGENS & RUNTIMES

Elisp (lógica principal — via Emacs)		
C (llama.cpp, performance crítica)		
Python 3.11+ (scripts auxiliares, análise)		
Shell bash (automação, deploy)		

CAMADA 1: HARDWARE

MÍNIMO: Raspberry Pi 5 (8GB) + SSD + fan		
RECOM.: x86_64 com 16GB RAM + GPU integrada		
IDEAL: MacBook Air M2+ ou Linux com GPU discreta		

7.2 Script de Instalação (INSTALL.org → shell blocks)



bash

```
#
# INSTALL.sh — gerado a partir de INSTALL.org
# Execute: bash scripts/install.sh
#
```

```
#!/bin/bash
set -euo pipefail

echo "🌀 Instalando web3-agent..."

# --- 1. Emacs 29+ ---
if ! command -v emacs &>/dev/null; then
  echo "📦 Instalando Emacs..."
  # Ubuntu/Debian:
  sudo apt-get install -y emacs-nox # headless
  # ou: sudo snap install emacs --classic
fi

# --- 2. llama.cpp ---
if ! command -v llama-server &>/dev/null; then
  echo "🤖 Compilando llama.cpp..."
  git clone https://github.com/ggerganov/llama.cpp /opt/llama.cpp
  cd /opt/llama.cpp && make -j$(nproc)
  sudo ln -sf /opt/llama.cpp/llama-server /usr/local/bin/llama-server
fi

# --- 3. Modelos GGUF ---
MODELS_DIR=~/.emacs.d/web3-agent/models
mkdir -p "$MODELS_DIR"
if [ ! -f "$MODELS_DIR/phi3-mini-q4.gguf" ]; then
  echo "📥 Baixando Phi-3 Mini Q4..."
  # ~2.4GB
  curl -L "https://huggingface.co/microsoft/Phi-3-mini-4k-instruct-gguf/resolve/main/Phi-3-mini-4k-instruct-q4.gguf" \
    -o "$MODELS_DIR/phi3-mini-q4.gguf"
fi

# --- 4. Python deps (para scripts auxiliares) ---
pip3 install --user requests pandas sqlite-utils

# --- 5. SQLite ---
sudo apt-get install -y sqlite3 libsqlite3-dev

# --- 6. Emacs packages (bootstrap via Emacs) ---
emacs --batch --load ~/.emacs.d/web3-agent/scripts/bootstrap-packages.el

# --- 7. Systemd service (Linux) ---
sudo cp infra/systemd/emacs-daemon.service /etc/systemd/system/
sudo cp infra/systemd/llamacpp.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable emacs-daemon llamacpp
sudo systemctl start emacs-daemon llamacpp

echo "✅ Instalação completa! Abra Emacs e execute: M-x web3-start"
```



8. Setup Experimental (Caso R\$300 / ONE+JEWEL)

8.1 Configuração para o Experimento



SETUP EXPERIMENTAL

OBJETIVO: Validar o sistema com capital real R\$300
TOKENS: ONE (Harmony) + JEWEL (DeFi Kingdoms)
DURAÇÃO: 30 dias (Fase 1: observação sem execução)
+ 30 dias (Fase 2: execução supervisionada)
+ 30 dias (Fase 3: semi-automático)

CONFIGURAÇÃO CONSERVADORA (padrão para R\$300):

perfil_risco: conservador	
capital_total: 300.00 BRL	
limite_auto_gate: 0.00 BRL	← FASE 1: zero auto
limite_human_gate: 300.00 BRL	← tudo vai para humano
max_exposicao_token: 40%	← máx 40% em 1 token
stop_loss: 15%	← alarme em -15%
hype_threshold: 0.30	← mindshare_cresc > 30%
preco_threshold: 0.50	← preco_cresc > 50%
timeout_gate: 3600 seg	← 1h para decidir
timeout_política: rejeitar	← sem resposta → não age

FASE 1 — OBSERVAÇÃO (dias 1-30):

→ daemon coleta dados	
→ LLM gera sugestões (todas vão para decisions.org)	
→ humano revisa e dá rating SEM executar	
→ sistema aprende o perfil real do humano	
→ ao final: análise de concordância humano×LLM	

FASE 2 — SUPERVISIONADA (dias 31-60):

→ gate humano para TODAS as ações	
→ limite_auto_gate aumenta para R\$5.00	
→ humano aprova/rejeita em tempo real	
→ RLHF dataset cresce com resultados reais	

FASE 3 — SEMI-AUTO (dias 61-90):

→ auto_gate até R\$15.00	
→ human_gate para ações maiores	
→ avaliação final: ROI vs. hold simples	

8.2 Métricas do Experimento



MÉTRICAS DE AVALIAÇÃO

MÉTRICAS PRIMÁRIAS:

ROI_sistema	vs	ROI_hold_simples	
taxa_acerto	=	alertas_corretos / total_alertas	
concordância	=	humano_aprovou / llm_sugериu	
latência_gate	=	tempo médio de resposta humana	

MÉTRICAS SECUNDÁRIAS:

custo_fees	=	total gasto em taxas de transação	
drawdown_max	=	maior queda relativa ao pico	
hype_precision	=	TP / (TP + FP) nos alertas de hype	
hype_recall	=	TP / (TP + FN) nos alertas de hype	

CRITÉRIOS DE SUCESSO (mínimo):

	taxa_acerto	>	65%	
	ROI_sistema	≥	ROI_hold_simples	
	custo_fees	<	3% do capital	
	drawdown_max	<	20%	
	Emacs daemon uptime	>	99% nos 90 dias	

9. Formalismos Gerais do Projeto

9.1 Álgebra de Gates



Gate G = (condição, timeout, política_default, log_entry)

Tipos de Gate:

- G_auto : condição automática (risco < limiar)
- G_human : requer presença humana
- G_time : dispara por tempo (scheduler)
- G_event : dispara por evento externo

Composição de Gates:

- G1 AND G2 : ambos devem ser aprovados
- G1 OR G2 : qualquer um suficiente
- NOT G1 : negação (bloqueia se aprovado)

Fluxo de Gate Humano:

resultado = G_human(ação, timeout, perfil)
resultado ∈ {APROVAR, REJEITAR, EDITAR(ação'), ADIAR(t')}

se resultado = ADIAR(t'):
 agendar G_human(ação, timeout, perfil) em t'

se timeout_expirou:
 resultado = política_default[perfil]
 conservador → REJEITAR
 moderado → MANTER_ESTADO
 arrojado → APROVAR

9.2 Gramática de Prompts (BNF)



```
<prompt> ::= <sistema> <contexto> <tarefa> <restrições>
<sistema> ::= "Você é" <papel> "operando como" <modo>
<papel> ::= "agente DeFi" | "analista de risco" | ...
<modo> ::= "processo contínuo" | "consultor pontual"
<contexto> ::= <timestamp> <mercado> <portfolio> <histórico>
<timestamp> ::= "Agora:" <iso8601>
<mercado> ::= "Mercado 24h:" <resumo_comprimido>
<portfolio> ::= "Portfolio:" <snapshot_json>
<histórico> ::= "Últimas decisões:" <n_decisões_resumidas>
<tarefa> ::= "Evento:" <tipo_evento> "\n" "Dados:" <payload>
            | "Pergunta:" <questão_livre>
<restrições> ::= "Regras:" <lista_regras> "Explique simples."
<lista_regras> ::= <regra> | <lista_regras> "\n" <regra>
<regra> ::= "max_exposição:" <pct>
            | "confirmar_se_valor_" <valor>
            | "perfil:" <tipo_perfil>
```

9.3 Modelo de Dados Org (Schema)



SCHEMA: decisions.org (RLHF dataset)

* DECISÃO [STATUS] :tags:
:PROPERTIES:
:ID: <uuid>
:timestamp: <[YYYY-MM-DD HH:MM]>
:ciclo: <n> ; número do ciclo do daemon
:evento_tipo: <string> ; "hype" | "rebalance" | "harvest"
:ação_llm: <string> ; ação sugerida pelo LLM
:confiança_llm: <0.0-1.0> ; score de confiança
:ação_humana: <string> ; ação efetivamente tomada
:delta_ação: <string> ; diferença (se editou)
:razão_humana: <string> ; por que divergiu (se divergiu)
:resultado_1d: <float> ; % de retorno em 24h
:resultado_7d: <float> ; % de retorno em 7d
:rating_humano: <1-5> ; avaliação posterior da decisão
:lora_ativa: <string> ; qual LoRA estava ativa
:prompt_hash: <sha256> ; para rastrear versão do prompt
:END:

** Contexto do LLM
<texto do contexto injetado no prompt>

** Resposta do LLM
<resposta bruta do LLM>

** Nota do Humano
<campo livre para o usuário anotar insights>

STATUS ∈ { PENDING | APPROVED | REJECTED | EDITED | EXPIRED }

🌟 10. Resultados Esperados

10.1 Por Fase





FASE SETUP (semana 1):

	Emacs daemon rodando 24/7	
	llama.cpp respondendo em < 5s	
	Primeiro ciclo completo de coleta → análise → alerta	
	Human gate funcional (y/n/e no buffer)	
	decisions.org populando com entradas	

FASE 1 — OBSERVAÇÃO (mês 1):

	30+ entradas em decisions.org	
	Taxa de concordância humano×LLM medida	
	Primeiro ajuste de threshold (auto-otimização)	
	Dashboard funcional mostrando energia financeira	

FASE 2 — SUPERVISIONADA (mês 2):

	Taxa de acerto de alertas > 60%	
	RLHF dataset com 100+ entradas e resultados reais	
	ROI ≥ hold simples (ou aprendizado sobre por quê)	
	Latência média de decisão humana documentada	

FASE 3 — SEMI-AUTO (mês 3):

	Sistema operando semi-autônomo com confiança	
	Dataset suficiente para fine-tuning de LoRA	
	Relatório final: o que funcionou, o que não	
	Artefato 09 completo: guia replicável	

10.2 Referências Teóricas Aplicadas



[1] Stallman, R. (1981). EMACS: The Extensible, Customizable Display Editor. MIT AI Lab. → base do paradigma daemon
APLICAÇÃO: emacs --daemon como processo $F=\emptyset$

[2] Ouyang et al. (2022). Training language models to follow instructions with human feedback. (InstructGPT/RLHF).
→ fundamento do RLHF inline
APLICAÇÃO: decisions.org como dataset de preferências

[3] Vaswani et al. (2017). Attention Is All You Need.
→ base dos LLMs que usamos localmente
APLICAÇÃO: Phi-3 como motor de inferência local

[4] Hu et al. (2021). LoRA: Low-Rank Adaptation.
→ ajuste fino eficiente
APLICAÇÃO: LoRAs por setor DeFi/NFT/regulação

[5] Milner, R. (1980). A Calculus of Communicating Systems.
→ formalismo de processos concorrentes (CCS)
APLICAÇÃO: formalismo $P=(S,\Sigma,\delta,s_0,\emptyset)$ do Artefato 01

[6] Kay, A. (1972). A personal computer for children of all ages. → Dynabook: computação como medium de pensamento
APLICAÇÃO: Emacs como "Dynabook do investidor DeFi"

[7] Berners-Lee et al. (2001). The Semantic Web.
→ ontologias como estrutura de conhecimento
APLICAÇÃO: ontologia do domínio (seção 1 deste artefato)

11. Metacríticas Finais do Artefato 02

-  **MC-01 — A Complexidade do Setup:** Este artefato pressupõe domínio de Emacs + Elisp + llama.cpp + Org-mode. Para um investidor de R\$300, isso pode ser barreira fatal. **Sugestão:** criar um instalador Docker one-liner com UI web opcional para quem não usa Emacs. O projeto fica bifurcado: web3-agent-emacs (poder total) e web3-agent-lite (acessível).
-  **MC-02 — RLHF com Dataset Pequeno:** 30-90 dias com R\$300 geram um dataset de 100-300 decisões. Isso é insuficiente para fine-tuning significativo de um LLM. **Sugestão:** usar o dataset para *ajuste de prompts e limiares* (que funciona com n pequeno), e reservar fine-tuning LoRA para quando o dataset superar 1000 entradas com resultados reais.
-  **MC-03 — Org-mode como Banco de Dados:** Para escala pequena (< 10k entradas), Org é excelente. Para escala maior, parsear Org-mode tem overhead significativo. **Sugestão:** manter Org para interface humana e auditoria, mas espelhar todas as entradas em SQLite (via emacsqli) para consultas analíticas. Dois formatos, uma fonte de verdade.
-  **MC-04 — O Paradoxo da Presença Humana:** Human-in-the-loop *bem implementado* é o oposto de automação: requer que o humano esteja *presente* e *atento*. Com R\$300, o custo de oportunidade do tempo humano pode superar o ganho. **Sugestão:** medir explicitamente tempo_gasto_humano vs. ganho_extra em relação a hold simples. Se razão < 1, reconsiderar o modelo. O projeto é também um experimento sobre onde a presença humana *realmente* agrega valor — e onde atrapalha.
-

💡 Filosofemas Finais do Artefato 02

💡 "Org-mode é o único banco de dados que o humano consegue ler sem um cliente especial. Isso não é limitação — é design."

💡 "Human-in-the-loop não é um backup para quando a IA erra. É o reconhecimento de que algumas decisões precisam de um ser que sente as consequências."

💡 "Um Elisp que nunca para é um pensamento que nunca dorme. O Emacs daemon é o inconsciente computacional do investidor."

💡 "RLHF inline é um diário de bordo que se converte em sabedoria. Cada y e cada n é uma frase nesse diário."

📌 Status & Próximos Passos

- 📄
✓
- ARTEFATO 01 ☒ Outline filosófico-técnico
- ARTEFATO 02 ☒ Spec completa (este documento)

- ARTEFATO 03 → agent.el: loop principal em Elisp (código real)
- ARTEFATO 04 → prompt-engine.el: RAG + templates adaptativos
- ARTEFATO 05 → gates.el + rlhf.el: human gates + RLHF inline
- ARTEFATO 06 → collector.el: fontes de dados (ONE/JEWEL)
- ARTEFATO 07 → dashboard.el: UI em Org-mode
- ARTEFATO 08 → mcp/server.el: integração MCP
- ARTEFATO 09 → Guia de deploy: R\$300 real, 90 dias

📌 Artefato vivo. Versão: 1.0 | Sessão: #02 | Série: IA×Energia×Web3 "Emacs é o único editor que pode ser o próprio sujeito do código que hospeda."